

PocketSmart AI: Your Smart Budget and Recommendation Assistant

Project Documentation

Date: 30 September 2026

Team ID: 6abc94497a3f72d0b9a67d15

Domain: Generative AI · Web Application · Budget Recommendation System

Project Description

PocketSmart AI is a Generative AI powered web application designed to help users make budget-aware decisions for planned purchases and occasions. The system accepts user requirements and budget-related inputs and provides personalized recommendations through three major planner modules: Home Interior Planner, Party Planner, and Jewelry Planner.

The application uses a FastAPI backend with Uvicorn, HTML/CSS/JavaScript and Jinja2-based templates for the frontend, and Google Gemini through the google-genai SDK for AI-assisted recommendation generation. Authentication and application data handling are organized through modules such as **auth.py** and **models.py**, while **gemini_utils.py** manages AI-related functionality.

Core Modules

- Home Interior Planner — generates budget-conscious interior planning recommendations.
- Party Planner — supports party planning with budget-oriented suggestions.
- Jewelry Planner — provides occasion-based jewelry recommendations.
- Authentication — supports controlled access to the application.
- AI Recommendation Engine — uses Google Gemini to generate personalized recommendation content.

Technical Architecture

PocketSmart AI follows a modular web-application architecture. The browser-based frontend collects user requirements, the FastAPI backend validates and processes requests, and the Gemini integration generates AI-assisted recommendations.

Layer	Technology / Component	Responsibility
Frontend	HTML, CSS, JavaScript, Jinja2	User interface, forms, navigation and dynamic presentation
Backend	Python, FastAPI, Uvicorn	Routing, request handling, validation and application logic
Authentication	auth.py	User authentication-related application flow
Models / Data	models.py	Application data/model definitions
AI Layer	Google Gemini + google-genai	Personalized recommendation generation
Static Assets	static/	CSS, JavaScript and supporting frontend assets
Templates	templates/	Jinja2 HTML templates

Architecture Flow

User → Web Interface → FastAPI Routes → Input Validation / Application Logic → Gemini AI Service → Structured Recommendation → Web Interface

Database / Persistence Note

The submitted project documentation should include an ER diagram if a persistent relational database is configured in the final deployment. The available project structure includes models.py, but this documentation does not invent database tables that are not explicitly verified.

Pre-requisites

- Python programming and virtual-environment setup.
- FastAPI and Uvicorn development knowledge.
- HTML, CSS and JavaScript fundamentals.
- Jinja2 template usage for server-rendered pages.
- Google Gemini / google-genai SDK familiarity.
- Git and GitHub for source-code version control.
- Environment-variable and API-key management.

Project Workflow

The development workflow is organized into milestones that move from understanding the problem to architecture, implementation, testing, documentation and demonstration.

Milestone	Major Activities
1. Problem & Feature Definition	Identify budgeting and recommendation needs; define Home Interior, Party and Jewelry planner requirements.
2. Architecture & Technology	Select Python, FastAPI, Uvicorn, Jinja2, HTML/CSS/JavaScript and Google Gemini.
3. Core Backend Development	Implement FastAPI application routes, authentication-related flow, models and AI utility functions.
4. Frontend Development	Build planner pages, forms, navigation, styling and dynamic interactions.
5. AI Integration	Connect the application to Gemini and construct recommendation requests based on user inputs.
6. Testing & Verification	Test authentication, planner workflows, invalid inputs, AI response handling and UI behaviour.
7. Documentation	Prepare project documentation, screenshots, setup information, limitations and future scope.
8. Demonstration	Prepare module-wise demo flow and explain architecture, implementation and results.

Expected End-to-End Flow

- User opens PocketSmart AI and accesses the appropriate planner.
- User provides requirements such as purpose, preferences and budget-related information.
- FastAPI receives and processes the request.
- The application prepares the information for AI-assisted recommendation generation.
- Gemini returns recommendation content.
- The application presents the generated suggestions in the web interface.
- The user reviews the suggestions and can refine the planning inputs.

Milestone 1: Problem Understanding and Architecture

This milestone establishes the purpose of PocketSmart AI and identifies how Generative AI can assist users in making budget-conscious planning decisions.

Activity 1.1: Define the Project Objective

- Create a single web application that assists users with common planning decisions.
- Use budget information as an important constraint while generating recommendations.
- Provide separate planner experiences for home interiors, parties and jewelry.
- Make recommendations understandable and easy to review.

Activity 1.2: Identify Target Users

- Individuals planning purchases or events within a specific budget.
- Users who need quick ideas rather than manually researching multiple alternatives.
- Users who want AI-assisted recommendations based on their stated preferences.

Activity 1.3: Define the High-Level Architecture

The frontend communicates with FastAPI routes. Backend logic validates inputs and coordinates the AI service. Gemini is used through the google-genai SDK, and the generated recommendation is returned to the application interface.

Activity 1.4: Select Development Environment

- Python for backend development.
- FastAPI and Uvicorn for the web service.
- Jinja2 templates with HTML/CSS/JavaScript for the interface.
- Google Gemini through google-genai for AI-assisted recommendations.

Milestone 1: Technology Setup and Security

The application requires a configured Python environment and secure handling of AI service credentials.

Activity 1.5: Environment Configuration

- Create a Python virtual environment for isolated dependencies.
- Install the project's dependency packages from the dependency configuration.
- Configure the Gemini API credential through environment configuration rather than hard-coding it in application source.
- Run the FastAPI application through Uvicorn during development.

Activity 1.6: Secure Configuration Practices

- Do not expose API keys in source code or public screenshots.
- Keep secret configuration outside the repository where possible.
- Use a safe example configuration file when documenting environment variables.
- Review authentication and input-handling logic before deployment.

Activity 1.7: Project Structure

Path	Purpose
main.py	FastAPI application entry point and route handling
auth.py	Authentication-related logic
models.py	Application model/data definitions
gemini_utils.py	Gemini integration and AI helper functions
templates/	Jinja2 HTML templates
static/	CSS, JavaScript and frontend assets

This structure keeps backend, authentication, model, AI integration and frontend assets separated for easier maintenance.

Milestone 2: Core Functionalities Development

The second milestone focuses on implementing the three principal planning experiences and the shared AI recommendation workflow.

Activity 2.1: Home Interior Planner

- Collect the user's interior planning requirements and budget context.
- Process the input through the backend.
- Generate AI-assisted ideas that respect the user's stated budget and preferences.
- Present the suggestions in a readable web interface.

Activity 2.2: Party Planner

- Collect party-related requirements such as occasion and planning preferences.
- Use the stated budget as a recommendation constraint.
- Generate practical suggestions for the user's party plan.

Activity 2.3: Jewelry Planner

- Collect occasion-related and preference-related inputs.
- Generate jewelry suggestions appropriate to the user's stated requirements and budget.
- Display the AI-generated recommendations clearly.

Activity 2.4: Authentication

The project includes an authentication module in **auth.py**. Authentication-related functionality helps control access to application features and supports a more structured user experience.

Milestone 2: Backend Implementation

The FastAPI backend acts as the central application layer between the browser interface and the AI recommendation service.

Activity 2.5: FastAPI Application

- Create the FastAPI application in `main.py`.
- Define routes for application pages and planner workflows.
- Receive user inputs from the frontend.
- Validate request data before processing.
- Invoke the appropriate recommendation logic.
- Return the generated result to the frontend.

Activity 2.6: AI Utility Layer

The `gemini_utils.py` module is used to keep Gemini-related operations separate from general route handling. This improves readability and makes the AI integration easier to maintain.

Activity 2.7: Model Layer

The `models.py` module provides a dedicated place for application model/data definitions. Separating models from routes helps keep the backend organized as the application grows.

Activity 2.8: Request-to-Response Flow

- Frontend sends planner input.
- FastAPI receives the request.
- Backend validates and prepares the information.
- Gemini utility constructs or sends the AI request.
- AI response is processed.
- The final recommendation is returned to the web page.

Milestone 3: Main Application Development

Milestone 3 concentrates on `main.py`, which serves as the central FastAPI application file.

Activity 3.1: Application Entry Point

`main.py` initializes the FastAPI application and connects the application's routes and supporting modules.

Activity 3.2: Route Handling

- Define the application's web routes.
- Connect requests to the appropriate planner functionality.
- Handle user input and pass it to the relevant processing layer.
- Return HTML pages or recommendation responses as required by the application flow.

Activity 3.3: Validation and Error Handling

- Check required inputs before sending information to the AI layer.
- Handle invalid or incomplete requests gracefully.
- Prevent application errors from producing unclear user experiences.
- Provide meaningful feedback when an AI request cannot be completed.

Activity 3.4: Module Integration

`main.py` works with `auth.py`, `models.py`, `gemini_utils.py`, `templates/` and `static/` so that the application remains modular instead of placing all functionality in one file.

Milestone 3: Gemini Integration

Google Gemini provides the Generative AI capability used by PocketSmart AI to produce personalized recommendation content.

Activity 3.5: AI Request Preparation

- Collect the relevant planner inputs.
- Combine user requirements with the selected planning context.
- Prepare a structured AI request that asks for useful, budget-aware recommendations.
- Send the request through the google-genai SDK.

Activity 3.6: AI Response Handling

- Receive the generated response from Gemini.
- Process the response before presenting it to the user.
- Handle unexpected or incomplete AI responses safely.
- Return the recommendation through the application workflow.

Activity 3.7: Recommendation Quality Considerations

- Recommendations depend on the information supplied by the user.
- Budget constraints should be included explicitly in the planner input.
- AI-generated suggestions should be reviewed by the user before making real purchasing decisions.
- The application should avoid exposing API credentials or internal implementation details.

Milestone 4: Frontend Development

The frontend provides the user-facing experience for accessing the planners and viewing AI-generated recommendations.

Activity 4.1: HTML and Jinja2 Templates

- Use templates/ for Jinja2-based HTML pages.
- Create structured forms for planner inputs.
- Display generated recommendations in clearly separated sections.
- Provide navigation between the main planner experiences.

Activity 4.2: CSS Styling

CSS files under static/ provide the visual styling of the application, including layout, spacing, typography, forms, cards and responsive presentation.

Activity 4.3: JavaScript Interaction

- Improve client-side interaction where required.
- Support dynamic form behaviour and result presentation.
- Provide user-friendly feedback during request processing.
- Keep the interface responsive across common screen sizes.

Activity 4.4: User Experience

The interface should make the three planner options easy to understand, keep required inputs visible, and present generated recommendations in a form that users can quickly read and compare.

Milestone 4: Planner User Interfaces

The three planner interfaces follow a common interaction pattern while collecting module-specific requirements.

Planner	User Interaction	Expected Output
Home Interior Planner	Enter interior requirements, preferences and budget-related information.	Budget-oriented interior planning recommendations.
Party Planner	Enter party/occasion requirements and planning constraints.	Party planning ideas and budget-oriented suggestions.
Jewelry Planner	Enter occasion, preference and budget-related requirements.	Occasion-based jewelry recommendations.

Common UI Flow

- Select or open a planner.
- Enter the required planning details.
- Submit the request.
- Show a processing state while the backend/AI service responds.
- Display the generated recommendation.
- Allow the user to revise inputs and request another recommendation.

Responsive Design

The frontend should remain usable on desktop and smaller screens. Layout and form elements are organized so that users can focus on the planner inputs and recommendation output without unnecessary navigation.

Milestone 4: Frontend-to-Backend Integration

The frontend and backend are connected through the application's FastAPI routes and template/request flow.

Integration Sequence

- The user submits planner information through the web interface.
- The request reaches a FastAPI endpoint.
- The backend validates and prepares the input.
- The AI utility layer communicates with Gemini.
- The generated response is returned to the application.
- The frontend renders the recommendation for the user.

Error Handling in the Interface

- Required fields should be validated before submission where possible.
- Backend validation should remain the authoritative validation layer.
- AI/API failures should produce a readable message instead of a raw exception.
- The user should be able to retry with corrected or revised inputs.

Maintainability

Keeping templates, static assets, authentication, models, AI utilities and the FastAPI entry point in separate modules makes future changes easier to manage and reduces unnecessary coupling.

Milestone 5: Local Deployment and Verification

The application should first be verified locally before any public or hosted deployment.

Activity 5.1: Prepare the Python Environment

- Install a supported Python version.
- Create and activate a virtual environment.
- Install project dependencies from the dependency file.
- Configure required environment variables for Gemini.

Activity 5.2: Start the FastAPI Application

Run the FastAPI application using Uvicorn with the project's configured entry point. The exact command should follow the final README/setup instructions supplied with the submitted source code.

Activity 5.3: Verify Core Workflows

- Open the application in a browser.
- Check authentication-related flow where enabled.
- Open each of the three planner modules.
- Submit valid planner inputs.
- Verify that recommendations are displayed.
- Test invalid or incomplete inputs and confirm that appropriate feedback is provided.

Milestone 5: Deployment Readiness

Before deployment, the application should be checked for configuration, dependency, security and reliability requirements.

Deployment Checklist

- All Python source files are present.
- templates/ and static/ directories are included.
- Dependency configuration is present.
- Required environment variables are documented without exposing secret values.
- Authentication flow is tested.
- All three planners are accessible.
- Gemini integration is configured correctly.
- Debug/development settings are not exposed unnecessarily in production.
- The final repository contains documentation and run instructions.

Public Deployment

If PocketSmart AI is deployed to a cloud provider, the final deployed URL should be recorded in the executable-files submission template. If the project is only demonstrated locally, the documentation should state that clearly rather than inventing a public URL.

Environment Security

API credentials must be kept private. Public repositories should contain only safe example configuration and must not expose active Gemini credentials.

Milestone 6: Testing and Verification

Testing verifies that PocketSmart AI behaves correctly across authentication, planner inputs, AI generation and frontend presentation.

Test Area	Verification
Authentication	Check valid and invalid authentication flows where implemented.
Home Interior Planner	Verify valid inputs generate an appropriate recommendation response.
Party Planner	Verify valid party planning inputs are processed and displayed.
Jewelry Planner	Verify occasion and preference inputs produce a recommendation response.
Input Validation	Check missing, invalid or incomplete inputs.
AI Integration	Verify Gemini requests and responses are handled correctly.
UI / Navigation	Verify planner navigation, forms and result presentation.
Error Handling	Verify API/service failures do not expose raw internal errors.

Performance Testing Note

A separate Performance Testing submission has been prepared for this project. Actual response-time, throughput, CPU and memory measurements should only be reported when captured from a performance-testing run; this documentation does not invent benchmark values.

Testing Scenarios

S.No	Scenario	Expected Result
1	Open the application and navigate to a planner.	Correct planner interface loads.
2	Submit valid Home Interior Planner input.	AI-assisted recommendation is returned and displayed.
3	Submit valid Party Planner input.	Party planning recommendation is returned.
4	Submit valid Jewelry Planner input.	Occasion-based recommendation is returned.
5	Submit incomplete input.	Validation or meaningful feedback is displayed.
6	Gemini service is unavailable.	Application handles the error without exposing secrets.
7	Navigate between planner pages.	Navigation works without broken routes.
8	Use the interface on a smaller screen.	Core content remains usable and readable.

Evidence

For the final submission, screenshots of successful planner outputs, relevant code, testing results and the running application can be inserted in this section or attached as supporting evidence. Only screenshots actually captured from the PocketSmart AI application should be used.

Project Structure and Source Code

The project is organized into backend modules and frontend resources.

Item	Description
main.py	Main FastAPI application and route definitions.
auth.py	Authentication-related functionality.
models.py	Application model/data definitions.
gemini_utils.py	Gemini AI integration/helper functions.
templates/	HTML/Jinja2 templates for application pages.
static/	CSS, JavaScript and other static frontend resources.

Code Organization Principles

- Keep route handling in the FastAPI application layer.
- Keep authentication concerns in auth.py.
- Keep model/data definitions in models.py.
- Keep Gemini-specific logic in gemini_utils.py.
- Keep presentation templates under templates/.
- Keep CSS and JavaScript under static/.

This organization supports readability, reusability and easier maintenance as new planner modules are added.

Exploring the Website's Web Pages

PocketSmart AI is designed around a planner-oriented web experience. The exact visual arrangement depends on the submitted frontend templates and static assets.

Home / Landing Experience

The home experience introduces PocketSmart AI and provides access to the available planning modules. Users can choose the type of planning assistance they need.

Home Interior Planner

The planner collects relevant interior requirements and budget context, then presents AI-generated recommendations intended to help the user plan within the stated constraints.

Party Planner

The party planner supports event-oriented planning and returns budget-conscious suggestions based on the user's inputs.

Jewelry Planner

The jewelry planner focuses on occasion-based recommendations and uses user preferences and budget information to guide the generated suggestions.

Input and Recommendation Experience

The planner interaction is designed to be simple: the user provides information, submits the request and reviews the generated recommendation.

Input Section

- Planner-specific requirements.
- Budget or spending constraints.
- Preferences relevant to the selected planning category.
- Additional context needed to make the recommendation useful.

Recommendation Section

- Generated suggestions are returned through the application's AI workflow.
- Recommendations should be presented in readable sections.
- Users can compare the suggestions against their original requirements.
- Users can revise their inputs when the first result does not meet their needs.

User Responsibility

PocketSmart AI provides AI-assisted suggestions rather than guaranteed purchasing, financial or professional advice. Users should verify prices, availability, quality and suitability independently before making real-world decisions.

Conclusion and Future Enhancements

PocketSmart AI demonstrates how Generative AI can be combined with a structured web application to support everyday planning decisions. The project integrates a FastAPI backend, Jinja2-based web templates, frontend technologies and Google Gemini to provide personalized recommendations across Home Interior, Party and Jewelry planning.

Current Capabilities

- Web-based access to multiple planning modules.
- Budget-aware recommendation workflow.
- Google Gemini integration through google-genai.
- Authentication-related functionality.
- Modular Python backend structure.
- HTML/CSS/JavaScript and Jinja2 frontend.

Potential Future Enhancements

- Cloud deployment with scalable infrastructure.
- Recommendation history and user personalization.
- Improved persistent storage and user profiles.
- Caching and performance optimization.
- Additional planning categories.
- Richer AI outputs and structured recommendation comparisons.
- Monitoring, logging and stronger production security.

Project Team and Final Documentation Summary

Team ID: 6abc94497a3f72d0b9a67d15

Project: PocketSmart AI: Your Smart Budget and Recommendation Assistant

Date: 30 September 2026

S.No	Team Member	Role
1	Sangavi B	Team Lead
2	Muhayya Yasmeen M	Team Member
3	Abinaya D	Team Member
4	Pawani M	Team Member
5	Padma Priya S	Team Member

Final Summary

PocketSmart AI brings together Generative AI and a practical web interface to help users plan purchases and occasions with budget constraints in mind. Its modular architecture separates the FastAPI application, authentication, models, Gemini integration and frontend resources, making the project suitable for continued development.

The completed documentation covers the project's concept, architecture, workflow, technology setup, core development, frontend integration, deployment readiness, testing approach, source-code organization, website experience, conclusion and future scope.

End of Project Documentation